Milestone Report: M0 — Foundations and Report Tool

Project: S-ERP (Sustainable Enterprise Resource Planning) DBMS
Milestone: M0 — Foundations and Report Tool
Branch: `m0-foundations`
Commit: `ca52619`
Date: 2026-10-29
Database Engines Tested: MySQL 8.4 & PostgreSQL 16

1. Summary

Milestone M0 establishes the engineering baseline, repository structure, dual-database runtime infrastructure, code verification pipeline, and automated reporting system for the Sustainable Enterprise Resource Planning (S-ERP) DBMS project.

The system is architected around Node.js 20 LTS, TypeScript (strict mode), Express 4, and direct database connectivity using native drivers (`mysql2` and `pg`) with **no ORM**. Both MySQL 8.4 and PostgreSQL 16 containers have been configured via `docker-compose.yml` , strictly enforcing a `512MB` RAM ceiling per container to adhere to host constraints. A live diagnostic endpoint (`GET /api/engines/status`) was implemented to actively verify host reachability, connection latency, and exact database engine versions on demand. Furthermore, the report toolchain was authored to compile this and future milestone reports into A4 PDF documents styled in conformance with the project's ledger-themed aesthetic palette and typographic rules, complete with git commit metadata and page numbering.

Research Paper to Product Mapping Table (from `plan.md` §1)

The conceptual foundations of this DBMS implementation originate from *Sustainable enterprise resource planning systems implementation: A framework development* (Chofreh et al., 2018). The table below outlines how core principles from the paper translate directly into concrete architectural elements of this system:

Paper Principle / Requirement	Implementation in S-ERP DBMS
Sustainability Paradigm (Environmental, Social, Economic)	Dedicated `sustainability_dimension` table; each indicator is classified into one of the three dimensions; dashboards and reports aggregate metrics by dimension.
Sustainability Indicators (Paper Fig. 5, Fernández-Sánchez & Rodríguez-López 2010)	Master seed catalog for `sustainability_indicator` covering 9 Environmental, 7 Social, and 5 Economic indicators.
Decisional Paradigm (Strategic, Tactical, Operational)	Categorized in `decision.decision_level`; four distinct user roles with dedicated interfaces and server-enforced access controls.
Project Management (5 process groups, 10 knowledge areas)	Structured management across `project`, `project_task`, `implementation_phase`, and `risk` tables.
S-ERP Roadmap (Pre-implementation, Implementation, Post-implementation)	Seeded phases representing ERP deployment lifecycle milestones in `implementation_phase`.
Integrated Database (Elimination of functional data silos)	Unified, 3NF relational schema shared across all modules with zero duplicate master entities.
Automated Sustainability Reporting (Eliminating manual spreadsheets)	First-class `report` entity with automatic calculation, archiving, and parameter-driven PDF export.
Triple Bottom Line Alignment	Decisions link processes, resources, and projects; dashboards render all three dimensions side by side.

2. Scope Check

Checklist Item from `plan.md` §8	Status	Rationale / Evidence
Repo, TypeScript config, ESLint, Prettier, `npm run check`	**Done**	`package.json`, `tsconfig.json`, `.eslintrc.cjs`, `.prettierrc` configured; `npm run check` passes cleanly.
`docker-compose.yml` with MySQL 8.4 & PostgreSQL 16 (memory-limited); `.env.example`	**Done**	Containers defined with `mem_limit: 512m`, persistent healthchecks, and non-conflicting host ports (MySQL `3307`, Postgres `5434`).
Copy paper and diagram image into `docs/source/`	**Done**	`DBMS_researchpaper.pdf` (1.2MB) and `er-usecase.png` (1.3MB) copied and verified in `docs/source/`.
`README.md` with clean-clone setup steps	**Done**	Comprehensive setup guide covering environment initialization, container launch, verification, and demo credentials.
Report tool: `docs/reports/_template.md`, `report.css`, `npm run report -- M<n>`	**Done**	Created report template, ledger styling tokens in CSS, and `scripts/report.ts` emitting A4 PDF with metadata footer.
Minimal server connecting to both engines; `GET /api/engines/status`	**Done**	Express server with `server/src/db/status.ts` executing live ping queries on both MySQL 8.4 and PostgreSQL 16.

3. What Was Built

3.1 Files Created / Modified

File Path	Purpose
`GEMINI.md`	Master ground rules and mandatory milestone gate workflow instructions.
`plan.md`	Master architectural plan, ER mappings, role definitions, and milestones.
`package.json`	Root package definition, dependencies, and strict verification scripts.
`tsconfig.json`	Root TypeScript configuration (ES2022, NodeNext resolution, strict mode).
`server/tsconfig.json`	Server-specific TypeScript project configuration emitting to `dist/server`.
`.eslintrc.cjs`	ESLint configuration for TypeScript syntax and style enforcement.
`.prettierrc`	Formatting rules (single quotes, 2 spaces, 100 char width).
`.env.example`	Template environment variables for ports, secrets, and dual-database credentials.
`.gitignore`	Git exclusion rules for node modules, secrets, build artifacts, and logs.
`docker-compose.yml`	Container orchestration for MySQL 8.4 and PostgreSQL 16 with 512MB RAM limits and healthchecks.
`docs/source/DBMS_researchpaper.pdf`	Upstream research paper source document.
`docs/source/er-usecase.png`	Upstream ER diagram and use case specification diagram.
`docs/reports/_template.md`	Standard markdown template for all milestone reports (M0–M9).
`docs/reports/report.css`	Ledger-themed print and screen stylesheet with CSS variables.
`scripts/report.ts`	Report compilation script converting Markdown into PDF with Git metadata footers.
`server/src/config/env.ts`	Zod-validated environment configuration loader.
`server/src/db/status.ts`	Database engine connectivity probe using `mysql2/promise` and `pg`.
`server/src/app.ts`	Express application instance defining health and engine status endpoints.
`server/src/index.ts`	HTTP server bootstrap listening on configured port.
`tests/server.test.ts`	Automated test suite verifying health and dual-engine status endpoints.
`README.md`	Clean-clone setup documentation, architecture summary, and credentials table.

3.2 API Endpoints

Method	Path	Role Required	Purpose
`GET`	`/api/health`	Public	Returns service liveness, name, and current timestamp.
`GET`	`/api/engines/status`	Public	Probes both MySQL 8.4 and PostgreSQL 16; returns host, port, database, reachability status, and engine version string. Returns HTTP 200 if both are reachable, 503 otherwise.

3.3 Screens / User Interfaces

- **API Status Output:** Structured JSON response returned by `GET /api/engines/status`.
- **Milestone PDF Generator:** Headless Chromium rendering engine generating A4 PDF reports with two-column ledger headers, table formatting, and running page footers.

---

4. Database Detail

4.1 Container & Resource Configuration

- **MySQL 8.4 Innovation / LTS:**
  - Image: `mysql:8.4`
  - Container Name: `serp_mysql`
  - Memory Limit: `512MB`
  - Host Port: `3307` (Container Port `3306`)
  - Volume: `serp_mysql_data`
  - Healthcheck: `mysqladmin ping -h 127.0.0.1 -u root -p$MYSQL_ROOT_PASSWORD`
- **PostgreSQL 16:**
  - Image: `postgres:16-alpine`
  - Container Name: `serp_postgres`
  - Memory Limit: `512MB`
  - Host Port: `5434` (Container Port `5432`)
  - Volume: `serp_postgres_data`
  - Healthcheck: `pg_isready -U $POSTGRES_USER -d $POSTGRES_DB`

4.2 Engine Differences Observed in M0

Dimension	MySQL 8.4	PostgreSQL 16
Driver Package	`mysql2/promise`	`pg` (`Client`)
Version Probe Query	`SELECT VERSION() as version`	`SELECT version()`
Response Format	Array of plain row objects `[ { version: '8.4.13' } ]`	QueryResult object with `.rows[0].version` containing full architecture/compiler banner
Default Port	`3306` (host mapped to `3307`)	`5432` (host mapped to `5434`)

---

5. How It Works

1. **Environment Initialization:** The backend loads and validates environment variables from `.env` using Zod (`server/src/config/env.ts`). If any configuration is malformed, the process immediately halts with descriptive schema errors.
2. **Container Health Monitoring:** Docker Compose initiates MySQL 8.4 and PostgreSQL 16 with memory constraints (`512MB`). Internal healthchecks periodically test database readiness.
3. **Dual Engine Probe:** When a client requests `GET /api/engines/status`:
   - An asynchronous probe connects to MySQL at `127.0.0.1:3307` and executes `SELECT VERSION()`.

- Concurrently, an asynchronous probe connects to PostgreSQL at `127.0.0.1:5434` and executes `SELECT version()`.
- Both connections use strict timeouts (3000ms) and automatically close their sockets upon completion.
- The aggregated status is returned with HTTP 200 (if both engines are active) or HTTP 503 (if either engine fails).

4. **Report Pipeline:** Executing `npm run report -- M0` reads `docs/reports/M0.md`, embeds current git revision and date, renders Markdown to HTML via `marked`, injects `docs/reports/report.css`, and renders `docs/reports/M0-foundations.pdf` via Puppeteer with headers and footers.

---

6. Screenshots & Visual Evidence



Figure 6.1: Live response from `GET /api/engines/status` displaying concurrent connectivity and version reporting for both MySQL 8.4.13 and PostgreSQL 16.15.

---

7. Tests and Evidence

7.1 Automated Check Suite (`npm run check`)

```
> s-erp-dbms@0.1.0 check
> npm run typecheck && npm run lint && npm run test

> s-erp-dbms@0.1.0 typecheck
> node node_modules/typescript/bin/tsc --noEmit && node node_modules/typescript/bin/tsc -p server/tsconfi

> s-erp-dbms@0.1.0 lint
> node node_modules/eslint/bin/eslint.js . --ext .ts,.tsx

> s-erp-dbms@0.1.0 test
> node node_modules/vitest/vitest.mjs run

 RUN  v2.1.9 /media/keerthivasan/NewVolume1/dbms

 ✓ tests/server.test.ts (2)
   ✓ S-ERP Server API (2)
     ✓ GET /api/health returns 200 with service information
     ✓ GET /api/engines/status returns structured status for both engines

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  22:06:54
   Duration  5.31s
```

7.2 Docker Compose Container Status (`docker compose ps`)

```
NAME           IMAGE                COMMAND                  SERVICE    CREATED          STATUS
serp_mysql     mysql:8.4            "docker-entrypoint.s…"   mysql      27 seconds ago   Up 26 seconds (
serp_postgres  postgres:16-alpine   "docker-entrypoint.s…"   postgres   27 seconds ago   Up 26 seconds (
```

7.3 Live MySQL 8.4 Engine Probe Response


```
{
  "engine": "mysql",
  "name": "SQL (MySQL 8.4)",
  "host": "127.0.0.1",
  "port": 3307,
  "database": "serp",
  "reachable": true,
  "version": "8.4.13"
}
```


7.4 Live PostgreSQL 16 Engine Probe Response


```
{
  "engine": "postgres",
  "name": "PostgreSQL 16",
  "host": "127.0.0.1",
  "port": 5434,
  "database": "serp",
  "reachable": true,
  "version": "PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit"
}
```


---

8. Deviations & Decisions

1. **Host Port Offsets (`3307` and `5434`):**
   - *Rationale:* The developer host environment had existing system services occupying default ports (`3306` for MariaDB and `5432` for PostgreSQL). Docker Compose was configured to map container port `3306` to host `3307`, and container port `5432` to host `5434`. This avoids host port collisions while ensuring the Dockerized environments match `MySQL 8.4` and `PostgreSQL 16` specifications exactly.
2. **Filesystem Execution Isolation (NTFS mount compatibility):**
   - *Rationale:* The workspace repository resides on a fuseblk/NTFS mounted volume with `noexec` permission defaults. Direct execution of native node binaries (such as `esbuild`) from inside `node_modules` is blocked by the OS kernel (`EACCES`). To ensure 100% build reliability without altering drive mount flags, all npm scripts execute via explicit node invocations (`node ./node_modules/...`) and `tsc` pre-compilation, while `esbuild` fallback paths are directed to user-space cache.
3. **Puppeteer Standalone Headless Engine:**
   - *Rationale:* To avoid dependency on system-level Chrome packages that may not be installed on clean machines, Puppeteer is configured with `chrome-headless-shell` residing in the user cache directory (`~/.cache/puppeteer`).

---

9. Known Issues and Limitations

- **Container Startup Period:** MySQL 8.4 requires approximately 15–20 seconds to complete internal data directory initialization before its healthcheck transitions to `healthy`. The backend probe gracefully handles this transient phase by returning `reachable: false` and HTTP 503 until the engine is fully online.
- **Single Host Deployment:** Both engines run on the local Docker network; distributed network replication between engines is intentionally omitted per `GEMINI.md` §5 (separate, independent data).

---

10. How to Run and Verify This Milestone

From a clean clone:

```
# 1. Copy environment template
cp .env.example .env

# 2. Launch database containers
docker compose up -d

# 3. Verify container health
docker compose ps

# 4. Install dependencies
npm install --ignore-scripts

# 5. Execute full verification suite (Typecheck, Lint, Tests)
npm run check

# 6. Build and start server
npm run build:server
npm run dev:server &

# 7. Probe engine status endpoint
curl -s http://127.0.0.1:3000/api/engines/status | jq .

# 8. Generate Milestone PDF Report
npm run report -- M0
```

11. Next Milestone

Target: Milestone M1 — Schema, Seed, and ER Conformance (Both Engines)
Deliverables in M1:

- DDL schemas for both engines: `db/mysql/schema.sql` and `db/postgres/schema.sql`.
- Migration and seeding runners: `scripts/migrate.ts` and `scripts/seed.ts` (with `--reset` support).
- Full domain seed dataset (1 org, 5 depts, 12 employees, 3 dimensions, Fig. 5 indicators, 12 months measurements, projects, tasks, risks, resources, processes).
- Live ER conformance validator: `scripts/er-check.ts` comparing live catalogs to ER specifications.
- Acceptance checks and M1 PDF report.

Prerequisite: Await explicit user authorization: `go M1`.